Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ «НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Домашнее задание №4

по дисциплине «Инженерия данных»

Выполнила:
Смирнова Анна Андреевна
ИСУ: 467514
Группа: J3311

Санкт-Петербург
2026

1 Цель работы

Освоение развёртывания, настройки и интеграции распределённого сервиса координации Apache ZooKeeper в приложение для обеспечения его отказоустойчивости, автоматического выбора лидера, восстановления после сбоев и гарантии согласованности данных при частичных отказах.

2 Развёртывание ансамбля ZooKeeper и проверка кворума (Этапы 1–2)

2.1 Конфигурация и проверка ролей узлов

В среде Docker Compose был поднят ансамбль из трёх серверов ZooKeeper 3.9.6 с уникальными директориями хранения данных, файлами `myid` (1, 2, 3) и клиентскими портами 2181, 2182, 2183. Межузловое взаимодействие настроено на портах 2888/3888.

После старта серверов опрос статуса подтвердил корректное распределение ролей в соответствии с алгоритмом Zab: узлы `zoo1` и `zoo2` перешли в режим **follower**, а узел `zoo3` стал **leader** (рис. 1).

```
PS C:\Users\LENOVO\Desktop\ITMO_labs\Data engineering vk\data-engineering-v
k> docker exec zoo1 zkServer.sh status; docker exec zoo2 zkServer.sh status
; docker exec zoo3 zkServer.sh status
ZooKeeper JMX enabled by default
Using config: /conf/zoo.cfg
Client port found: 2181. Client address: localhost. Client SSL: false.
Mode: follower
ZooKeeper JMX enabled by default
Using config: /conf/zoo.cfg
Client port found: 2182. Client address: localhost. Client SSL: false.
Mode: leader
ZooKeeper JMX enabled by default
Using config: /conf/zoo.cfg
Client port found: 2183. Client address: localhost. Client SSL: false.
Mode: follower
```

Рис. 1: Распределение ролей в ансамбле: два последователя и один лидер

2.2 Моделирование отказов: кворум и блокировка split-brain

Кворум для системы из $N = 3$ узлов рассчитывается как $Q = \lfloor 3/2 \rfloor + 1 = 2$.

1. **Отказ одного узла (`docker stop zoo1`):** в строю остались 2 сервера из 3 ($2 \geq 2$, кворум сохранён). Ансамбль выполнил перевыборы: лидерство перешло к `zoo2`, сервис продолжил штатно обслуживать запросы (рис. 2).

2. **Отказ двух узлов (`docker stop zoo2`):** остался лишь 1 узел `zoo3` ($1 < 2$, кворум утерян). Сервер вернул ошибку «*Error contacting service. It is probably not running*» и заблокировал операции, предотвращая рассинхронизацию данных (рис. 3).

```
PS C:\Users\LENOVO\Desktop\ITMO_labs\Data engineering vk\data-engineering-v
k> docker exec zoo2 zkServer.sh status; docker exec zoo3 zkServer.sh status

ZooKeeper JMX enabled by default
Using config: /conf/zoo.cfg
Client port found: 2182. Client address: localhost. Client SSL: false.
Mode: leader
ZooKeeper JMX enabled by default
Using config: /conf/zoo.cfg
Client port found: 2183. Client address: localhost. Client SSL: false.
Mode: follower
```

Рис. 2: Отказ 1 сервера: кворум сохранён, zoo2 успешно стал лидером

```
PS C:\Users\LENOVO\Desktop\ITMO_labs\Data engineering vk\data-engineering-v
k> docker exec zoo3 zkServer.sh status
ZooKeeper JMX enabled by default
Using config: /conf/zoo.cfg
Client port found: 2183. Client address: localhost. Client SSL: false.
Error contacting service. It is probably not running.
```

Рис. 3: Отказ 2 серверов: потеря кворума и защитная блокировка узла zoo3

2.3 Проверка жизненного цикла эфемерного узла

В интерактивной консоли `zkCli` был создан эфемерный узел `/test_ephemeral` со значением `my_ephemeral_value`. После закрытия сессии командой `quit` и разрыва TCP-соединения узел был автоматически удалён ансамблем ZooKeeper, что подтверждает привязку времени жизни эфемерного узла к сессии клиента (рис. 4).

```
WATCHER::

WatchedEvent state:SyncConnected type:None path:null zxid: -1
[zk: localhost:2181(CONNECTED) 0] Created /test_ephemeral
[zk: localhost:2181(CONNECTED) 1] my_ephemeral_value
[zk: localhost:2181(CONNECTED) 2]
WATCHER::
```

(a) Создание узла в активной сессии

```
WATCHER::

WatchedEvent state:SyncConnected type:None path:null zxid: -1
[zk: localhost:2181(CONNECTED) 0] Node does not exist: /test_ephemeral
[zk: localhost:2181(CONNECTED) 1]
```

(b) Автоудаление после закрытия сессии

Рис. 4: Демонстрация работы эфемерного узла в клиенте zkCli

3 Приостановка лидера без защиты по эпохе (Этап 3, FENCING=0)

На языке Python с использованием библиотеки `kazoo` был запущен координатор аналитики вакансий (поиск пар навыков из ДЗ №3) с таймаутом сессии 10 секунд и фиксацией чекпоинтов каждые 50 000 строк.

Ход эксперимента: Были запущены два экземпляра: `coord1` (Лидер) и `coord2` (Standby). На 50 000-й строке процесс `coord1` был приостановлен системным сигналом на 40 секунд.

- Через 10 секунд ZooKeeper зафиксировал истечение сессии и удалил эфемерный лок.
- **Измеренное время смены лидера: 11.74 секунды**.
- `coord2` стал новым лидером, подхватил расчёт с чекпоинта (с 50 001-й строки), завершил его и сохранил результат:

  ```
  [coord2] coord2 записал результат (версия 1)
  ```

- Спустя 40 секунд старый лидер `coord1` возобновил работу, досчитал строки и перезаписал данные:

  ```
  [coord1] coord1 записал результат (версия 2)
  ```

В журнале зафиксированы **две строки «записал результат»**, а итоговый указатель в ZooKeeper получил версию 2. Результаты вычислений обоих координаторов полностью совпали с ДЗ №3 (Таблица 1).

Таблица 1: Верификация контрольных чисел: ДЗ №3 vs Координатор ДЗ №4

Навык / Пара навыков	**ДЗ №3 (MapReduce)**	**ДЗ №4 (Координатор)**
1С программирование	8 278	8 278
1С: Предприятие 8	4 414	4 414
1С: Бухгалтерия	2 781	2 781
1С: Управление Торговлей	2 530	2 530
ERP-системы на базе 1С	2 335	2 335
1С прогр. + 1С: Предприятие 8	3 540	3 540
1С прогр. + 1С: Бухгалтерия	2 172	2 172
1С прогр. + 1С: Управление Торговлей	1 953	1 953
1С прогр. + Обновление конфигурации 1С	1 895	1 895
1С прогр. + ERP-системы на базе 1С	1 764	1 764

4 Приостановка лидера с защитой по эпохе (Этап 4, FENCING=1)

Эксперимент был повторён с включённым флагом `FENCING=1`. Защита реализована на базе оптимистической блокировки ZooKeeper: при публикации проверяется версия целевого znode (`version=expected_version`).

Результат выполнения:

- Время переключения на запасного координатора составило **11.25 секунды**.
- Запасной координатор `coord2` завершил расчёт и успешно опубликовал результат:

```
[coord2] coord2 записал результат (версия 1, указатель от coord2)
```

- Проснувшийся через 40 секунд «зомби-лидер» `coord1` попытался отправить свои данные с устаревшей версией 0, но операция была атомарно отклонена сервером:

```
[coord1] ошибка публикации: BadVersionError: версия 0 устарела!
```

В хранилище остался ровно один валидный результат от `coord2`, а коллизия двух лидеров была успешно предотвращена.

5 Ответы на теоретические вопросы (Этап 5)

5.1 Вопрос 1: Почему серверов три, а не два и не четыре?

Выбор количества серверов определяется правилом кворума — строгого большинства участников ($\lfloor N/2 \rfloor + 1$). Для двух серверов кворум равен двум: отказ хотя бы одной машины или сетевой разрыв между ними полностью парализует работу, то есть система имеет нулевую отказоустойчивость. Для четырёх серверов кворум составляет три узла, поэтому такой кластер способен пережить падение лишь одного сервера — ровно столько же, сколько и ансамбль из трёх машин (где достаточно двух работающих). При этом четвёртый узел не повышает надёжность, но замедляет подтверждение транзакций по протоколу Zab и требует лишних ресурсов. Поэтому ансамбль всегда формируют из нечётного числа узлов, где тройка является минимальной самодостаточной конфигурацией.

5.2 Вопрос 2: Почему защиту по эпохе проверяет получатель записи, а не ZooKeeper?

Причиной является специфика частичных отказов. Старый лидер может кратковременно зависнуть из-за паузы сборщика мусора (GC) или задержек в сети. ZooKeeper фиксирует истечение таймаута сессии, считает лидера отключившимся и выбирает нового, открывая новую эпоху. Однако после разморозки «зомби-лидер» не знает о потере статуса и отправляет свои данные напрямую в целевое хранилище. Сам ZooKeeper является внешним координатором, не выступает прокси-сервером для прикладного трафика и физически не может перехватить сетевые пакеты, летящие в базу данных. Поэтому проверять номер эпохи обязана сама принимающая сторона: видя токен с меньшим номером версии, хранилище атомарно отклоняет устаревшую запись.

6 Заключение

В ходе работы развёрнут ансамбль ZooKeeper из трёх серверов. Практически подтверждена устойчивость к отказу одного узла и защитная блокировка при потере кворума. Реализован координатор вычислений, измерен интервал failover (≈ 11.5 с), подтверждена точность расчётов с чекпоинта и продемонстрирована работа механизма защиты по эпохе с возникновением `BadVersionError`.

Репозиторий с конфигурацией и логами запусков:
https://github.com/Anchik0000/data-engineering-vk